Nowcasting the economy

Group Members:

Bryce Tan Jing Kai (A0272764W)

Chin Chen Shao Javier (A0272898E)

Gan Zhi Yu Charlene (A0282072J)

Melanie Tan Yong En (A0277113J)

Owen Lim Wen Xuan (A0272606E)

Shannon Kwok En Yi (A0281617B)

Tan Sze Ping (A0286558J)

Vanisha Muthu (A0282716Y)

Part I (prepared by the entire group)

1.0 Project Overview

Official GDP figures are released with a substantial lag, limiting their usefulness for real-time economic assessment and causing policy decisions to be reactive rather than forward-looking, particularly during periods of economic turning points. The primary goal of this project is to develop a real-time nowcasting system for current-quarter GDP growth, enabling more timely and informed decision-making.

To achieve this, we develop an interactive web-based application that generates and continuously updates GDP nowcasts using high-frequency macroeconomic indicators. By incorporating newly released monthly data, the system produces dynamic estimates of quarterly GDP growth, allowing users to track evolving economic conditions within the quarter. The application also supports comparison against benchmark models, enabling users to evaluate predictive performance alongside current estimates.

The primary users of this application are monetary policymakers and economic analysts, particularly within central banks and financial institutions. These users require timely and reliable signals of economic activity to detect early signs of downturns and identify turning points in the business cycle. The system therefore serves as a decision-support tool that complements traditional macroeconomic analysis and enhances real-time monitoring capabilities.

The project leverages a combination of statistical and machine learning tools within a modular pipeline. Data is sourced from the Federal Reserve Economic Data (FRED) database, specifically the FRED-MD (monthly) and FRED-QD (quarterly) datasets. Data processing and modeling are implemented in Python using libraries such as *pandas*, *NumPy*, and *statsmodels*, while the front-end interface presents results through interactive visualisations.

GitHub Repository: https://github.com/shannon-kwok/DSE3101-Proj/tree/main
Deployment Link : https://dse3101-proj-nncf8h3qkwgetp9nj6vzbc.streamlit.app/

2.0 Overall Design

The web application is designed using a modular client–server architecture, separating the front-end interface from the back-end data processing and modeling components. This structure ensures scalability, flexibility and ease of maintenance, which allows each component to be developed and improved independently.

The front-end serves as the user interface, where users interact with the application to generate and explore GDP nowcasts. It is responsible for capturing user inputs (e.g., time period selection or model configuration), sending requests to the back-end, and displaying outputs through interactive visualisations such as time-series plots and model comparison charts.

The back-end handles data processing, model execution, and data management. Upon receiving a request from the front-end, it retrieves the relevant macroeconomic data, applies preprocessing steps, runs the nowcasting models, and returns the generated predictions. The back-end is structured as a modular pipeline, allowing different models (e.g., bridge models and autoregressive benchmarks) to be executed and compared efficiently. Communication between the front-end and back-end is managed through API calls, ensuring a smooth and responsive exchange of data and results.

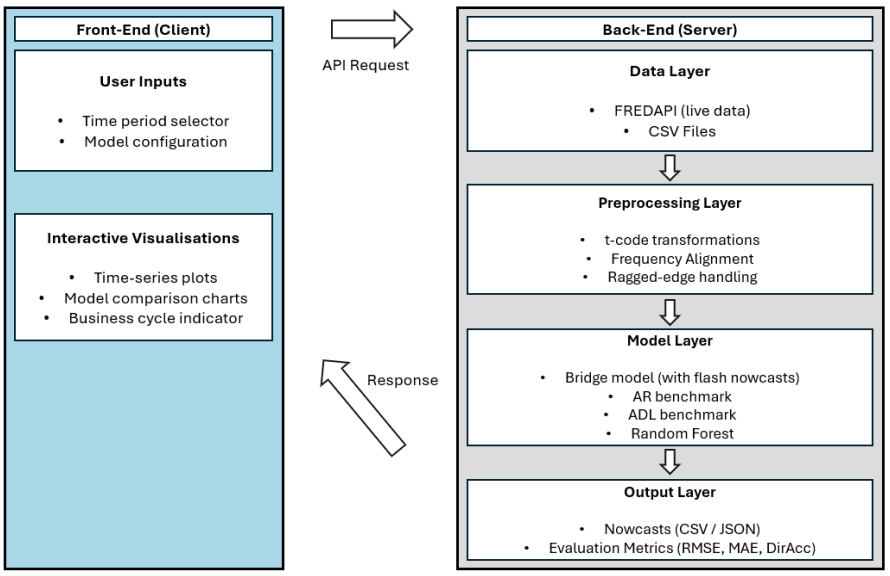


Figure 1 : End-to-End Client-Server Architecture

3.0 Data and methodology

The analysis uses two key datasets from the Federal Reserve Economic Data (FRED) repository. FRED-MD (McCracken & Ng, 2015) provides approximately 150 monthly macroeconomic indicators, including industrial production, labor market measures, consumer sentiment, financial variables, and exchange rates, which serve as the primary predictors for the nowcasting models. The dataset is documented in the working paper *FRED-MD: A Monthly Database for Macroeconomic Research* (McCracken & Ng, 2015), available at https://doi.org/10.20955/wp.2015.012.

Complementing this, FRED-QD (McCracken & Ng, 2020) provides quarterly macroeconomic indicators, with real GDP (GDPC1) as the primary series of interest. Quarterly GDP growth derived from GDPC1 serves as the target variable for the nowcasting models, allowing the integration of high- and low-frequency information to generate timely estimates of economic activity. The dataset is documented in the working paper *FRED-QD: A Quarterly Database for Macroeconomic Research* (McCracken & Ng, 2020), available at https://doi.org/10.20955/wp.2020.005.

The data preprocessing pipeline prepares high-frequency macroeconomic indicators and quarterly GDP for nowcasting by ensuring stationarity, handling missing data, and aligning mixed frequencies. The workflow consists of four stages: data loading, transformation, aggregation, and feature selection. Transformations such as differencing and logarithmic adjustments are applied to reduce spurious correlations, while ragged-edge data are addressed through imputation or truncation. Monthly indicators are aggregated to match quarterly GDP, and relevant predictors are selected based on economic and statistical considerations. The project employs a bridge model framework that links monthly indicators to quarterly GDP growth, enabling real-time nowcasts using partial within-quarter information. Its performance is evaluated against an autoregressive (AR) benchmark based solely on past GDP, highlighting the gains from incorporating high-frequency data.

Part II (to be prepared by the back-end team)

4.0 Data Pre-Processing and Transformation

4.1 Data Transformation and Stationarity

To ensure the suitability of the data for time-series modelling, all variables are transformed to achieve stationarity. Macroeconomic time series often exhibit trends, seasonality or unit roots, which can lead

to spurious regression results if left untreated. Following the methodology of McCracken and Ng (2015, 2020), each variable in both FRED-MD and FRED-QD is associated with a transformation code that specifies the appropriate transformation. These transformations include taking first or second differences, logarithms, or combinations thereof. (Refer to Figure 2)

These transformations are applied programmatically to each series, ensuring consistency with the recommended preprocessing standards in the FRED databases. In particular, log-differencing is widely used for macroeconomic variables such as output and prices, as it approximates percentage growth rates and stabilizes variance. For the target variable, real GDP (GDPC1), the same transformation procedure is applied using its corresponding t-code from FRED-QD. The resulting series represents quarterly GDP growth, which serves as the dependent variable in the nowcasting models.

4.2 Frequency Alignment and Aggregation

A key feature of nowcasting is the use of mixed-frequency data, where high-frequency monthly indicators are used to predict lower-frequency target variables such as quarterly GDP. Through *aggregate_to_quarterly()* function we address the mismatch in data frequency between the MD and QD datasets, converting the MD data set into quarterly frequency by first mapping each observation to its corresponding quarter and aggregating using a group-by operation: *monthly_df['quarter'] = monthly_df.index.to_period('Q')*, followed by *quarterly = monthly_df.groupby('quarter').mean()*
Each monthly observation is first mapped to its corresponding quarter using a *PeriodIndex*, after which variables are aggregated using simple averages across the three months in each quarter. This produces a smoothed quarterly representation of the underlying indicators. After aggregation, the predictors are merged with the GDP growth series using an inner join: *data = monthly_q.join(gdp_series, how='inner').dropna()*. This ensures that only observations with both predictor and target data are retained, resulting in a fully aligned dataset for model estimation.

To account for large exogenous shocks, such as the COVID-19 pandemic, a dummy variable is optionally included: *data['covid_dummy'] = ((data.index >= start_q) & (data.index <= end_q)).astype(int)*. This variable equals 1 for the period 2020Q1–2020Q4, allowing the model to capture the structural break associated with the pandemic.

4.3 Cleaning and Sample Construction

The sample period is restricted to 1960Q1–2020Q2 to mitigate potential structural breaks and ensure consistent data coverage across all series. This ensures that monthly predictors and quarterly GDP are aligned within a stable estimation window, facilitating robust model estimation.

In this project, missing values are initially retained during the transformation stage to avoid unnecessary data loss. However, since subsequent steps, such as quarterly aggregation and LASSO-based feature selection, require sufficiently complete data, variables with excessive missingness are removed before model estimation.

Specifically, predictors with more than 5% missing observations are excluded. This ensures sufficient time-series coverage for reliable transformation, aggregation, and regression analysis. By removing these variables, the dataset becomes more balanced and suitable for LASSO feature selection and improves the robustness of the nowcasting framework. (Refer to Figure 3)

In the main script, this is implemented as: *vars_to_drop = ['ACOGNO', 'UMCSENTx', 'TWEXAFEGSMTHx', 'ANDENOx', 'VIXCLSx']* and *MD_trans = MD_trans.drop(columns=vars_to_drop, errors='ignore')*. These steps collectively improve computational efficiency, reduce dimensionality, and produce a cleaner, more reliable dataset for downstream feature selection and nowcasting. (Refer to Figure 4)

4.4 Feature Selection using LASSO

Given the high dimensionality of the FRED-MD dataset, careful feature selection is essential to construct a reliable nowcasting model and avoid introducing noise that could degrade out-of-sample performance. We employ Regularized LASSO (rlasso) via the *hdmpy* package, implemented in *feature_selection.py* through the *select_features_rlasso()* function. This approach simultaneously performs coefficient shrinkage and variable selection, assigning zero coefficients to irrelevant predictors while retaining those with genuine explanatory power.

The target variable for the nowcasting model is quarterly real GDP growth. To ensure a consistent penalty across all predictors, observations with missing values are removed, and remaining variables are standardized to zero mean and unit variance. This prevents variables with larger numerical ranges from being disproportionately penalized. Certain predictors, such as the COVID-19 dummy, can be retained explicitly using the *exclude_cols* argument, ensuring their inclusion regardless of regularization.

In our implementation, LASSO was used to obtain a fixed set of predictors from the training data. This provides a stable and transparent specification for the subsequent regressions, while reducing the dimensionality of the original macroeconomic dataset.

To further enhance model stability and address potential multicollinearity among selected predictors, two diagnostic checks are applied:

1. **Pairwise correlation screening**: After LASSO selection, features with absolute correlations above 0.9 are flagged via *get_high_correlation_pairs()*, allowing for further pruning if needed.
2. **Variance Inflation Factor (VIF) analysis**: Optionally, VIFs are calculated using *calculate_vif()* to detect predictors that may induce multicollinearity in the bridge regression. Features with high VIF values can be considered for removal to stabilize regression coefficients.

The final 6 selected variables are as follows:

Group 1: Output and Income

1. W875RX1: Real Personal Income excluding Transfer Receipts
2. IPDMAT: IP: Durable Materials

Group 2: Labour Market

1. DMANEMP: All Employees: Durable Goods
2. UNRATE: Civilian Unemployment Rate
3. UEMP15T26: Civilians Unemployed for 15-26 Weeks

Group 4: Consumption, order and inventories

1. DPCERA3M086SBEA: Real Personal Consumption Expenditures

The groupings are based on the FRED-MD (McCracken & Ng, 2015) classification. Considering how LASSO pulled 3 variables from Group 2, but none from Housing (Group 3), Money (Group 5) or Prices (Group 7), this suggests that the labour market offers greater predictive power than other common macro categories. Also, “DMANEMP” and “UEMP15T26” are leading indicators, while “W875RX1”, “IPDMAT” and “DPCERA3M086SBEA” are coincident indicators. Coincident variables provide a stable measure of how the economy is doing right now, while leading variables are extremely valuable for imputing missing monthly values when handling the ragged data edges. Leading variables also enhance the model’s responsiveness to any sudden cyclical shifts in the economy, which is not yet captured by coincident variables.

5.0 Modeling and Nowcasting Framework

5.1 Overview

This section outlines the modeling framework used to generate real-time nowcasts of quarterly GDP growth. The approach combines mixed-frequency econometric modeling with machine learning benchmarks, evaluated under a realistic expanding-window framework. The core methodology is based on a bridge model, which links high-frequency monthly indicators to quarterly GDP. A key challenge in this setting is the presence of ragged-edge data, where some monthly indicators are unavailable for the most recent periods. This is addressed using autoregressive (AR) models to impute missing observations. Model performance is evaluated using an out-of-sample expanding window approach, which ensures that only information available at each point in time is used, thereby avoiding look-ahead bias.

5.2 Bridge Model

The bridge model relates quarterly GDP growth to selected monthly indicators aggregated to quarterly frequency. To address the ragged edge problem, we fit an autoregressive (AR) model to each indicator. Lag lengths are selected via the Akaike Information Criterion (AIC), and each AR model is used to forecast the missing values required to complete the dataset for the missing months in the quarter. The filled monthly series are then aggregated to quarterly frequency using simple averages. Finally, the bridge regression is estimated using Ordinary Least Squares (OLS) to produce the current quarter nowcast.

This entire sequence is repeated under an expanding-window framework across all forecast periods. While the bridge model coefficients and AR imputation models are re-estimated recursively under an expanding window, we do not constantly rerun LASSO to do feature selection. Rather, the initial set of indicators chosen by LASSO is used throughout. This is to avoid look-ahead bias, as rerunning LASSO feature selection repeatedly would mean that the algorithm has access to the full historical relationship between regressors and the response. This also ensures that the set of predictors remains consistent across all forecast horizons, maintaining model interpretability.

5.3 Flash Nowcasting

To better reflect real-time forecasting conditions, the bridge model is extended using a flash nowcasting framework, which explicitly accounts for the sequential release of monthly data within a quarter. In practice, macroeconomic indicators are not released simultaneously, leading to incomplete information when estimating GDP for the current quarter. To simulate this realistic information flow, three flash scenarios are constructed:

- Flash 1: Only the first month of the quarter is observed
- Flash 2: The first two months are observed
- Flash 3: All three months are observed

For each flash scenario, the monthly dataset is modified such that only the available observations are retained, while the remaining months are treated as missing. These missing observations are then imputed using the same AR-based ragged-edge procedure described earlier.

Specifically, for each selected indicator:

- Observed monthly values are retained up to the flash cutoff
- Missing months within the quarter are masked
- AR(p) models are used to forecast the missing values
- The completed monthly dataset is aggregated to a quarterly frequency

This produces three different versions of the predictor matrix for each quarter, corresponding to different levels of information availability. The flash nowcasting framework allows us to evaluate how predictive accuracy evolves as more data becomes available within the quarter. In particular, Flash 2 is

used as the primary nowcast, as it represents a realistic mid-quarter information set. This approach ensures that the nowcasting exercise reflects real-world data constraints and avoids look-ahead bias, providing a more accurate assessment of model performance.

5.4 Benchmark Models

Three benchmark models (AR, ADL, and Random Forest) are implemented to provide reference points for comparison. A key design consideration is ensuring fair comparability, where all models use only information available at time $t - 1$. This is achieved through explicit construction of lagged variables.

5.4.1 Autoregressive (AR) Model

The AR model serves as a simple baseline, using only past values of GDP growth to generate forecasts. This model captures persistence in GDP growth but does not incorporate any information from high-frequency indicators. The AR model uses only lagged GDP values, with lag order selected via AIC.

5.4.2 Autoregressive Distributed Lag (ADL) Model

The ADL model extends the AR framework by including lagged macroeconomic predictors, including credit spread, unemployment, and housing indicators, alongside lagged GDP. This allows it to capture both temporal dynamics and the influence of key economic indicators. However, it operates purely at the quarterly level and does not account for mixed-frequency data or ragged-edge issues.

5.4.3 Random Forest (RF) Benchmark

The RF model is included as a nonlinear forecasting benchmark to capture complex relationships between predictors and GDP growth. To ensure a fair comparison, the model uses only lagged quarterly predictors and GDP lags, avoiding the use of contemporaneous information.

5.4.4 Model Specification

Model	Predictors
Bridge (Flash 2)	GDP_growth_lag1, GDP_growth_lag2, + covid_dummy + 6 LASSO selected indicators
AR	GDP growth lags (order selected by AIC, max 8)
ADL	GDP_growth_lag1, GDP_growth_lag2 + (BAA-AAA)_lag1, UNRATE_lag1, HOUST_lag1
RF	GDP_growth_lag1, GDP_growth_lag2 + Lagged 6 LASSO indicators (by 1 quarter)

5.5 Evaluation Metrics

Model performance is evaluated using an out-of-sample expanding window framework, where models are re-estimated each quarter using only past data. This ensures a realistic nowcasting setting without data leakage. For the bridge model, evaluation is conducted using the flash nowcasting framework, ensuring that predictions are generated using partial information sets rather than fully observed data. In contrast, benchmark models are evaluated using lagged quarterly predictors. This ensures that all models are compared under consistent real-time information constraints, avoiding data leakage.

Three metrics are used:

- RMSE: Penalises large errors and serves as the main measure of accuracy
- MAE: Measures average error magnitude, less sensitive to outliers

- Directional Accuracy: Measures how often the model correctly predicts the direction of GDP growth

5.6 Results

Model	RMSE	MAE	Directional Accuracy
Bridge (Flash 2)	**0.0066**	**0.0044**	**0.925**
AR	0.0111	0.0052	0.875
ADL	0.0100	0.0054	0.875
RF	0.0103	0.0048	0.888

The bridge model outperforms all benchmarks, achieving roughly 50% lower RMSE than AR and ADL. While Random Forest improves over linear benchmarks, it still underperforms compared to the bridge model. This highlights the value of incorporating higher frequency monthly indicators rather than relying solely on lagged quarterly data to improve nowcast accuracy.

5.7 Live Nowcasting

The live nowcasting framework is designed to generate real-time estimates of quarterly GDP growth using the latest available monthly macroeconomic indicators. Monthly data and quarterly GDP are first retrieved via the API and preprocessed to ensure consistency and stationarity. In particular, the function *transform_series()* applies the FRED transformation codes to each indicator, while *aggregate_to_quarterly()* converts the monthly data into quarterly predictors aligned with GDP. GDP growth is defined as the log first difference of real GDP, which provides an approximate percentage interpretation and ensures suitability for regression-based models.

To address the missing monthly indicators, the framework uses *fit_ar_models()* together with *fill_ragged_edge()* to estimate missing monthly values via univariate autoregressive models. These imputed values are then aggregated to construct a complete set of quarterly predictors for each "flash" stage, allowing the bridge model to produce progressively updated nowcasts as more data becomes available.

The main predictive model is the bridge regression, implemented using *fit_bridge_model(),* which relates GDP growth to a set of selected indicators and lagged GDP terms. In addition, benchmark models are included for comparison: an ADL model, estimated via fit_adl_benchmark(), and a univariate AR model fitted using *fit_ar_benchmark()*. These models provide alternative specifications based on different assumptions about the role of past GDP and macroeconomic indicators.

An important enhancement in the live pipeline is the ability to nowcast multiple consecutive quarters even when the most recent GDP observation is unavailable. This is achieved through a recursive nowcasting approach. After generating a nowcast for the first missing quarter (e.g. 2026Q1), the predicted GDP growth is inserted back into a temporary GDP series and used as a lagged input for the next quarter (e.g. 2026Q2). This ensures that required variables such as lagged GDP growth remain available for model estimation. Notably, this is not an imputation using an external model, but rather a consistent extension of each model's own predictions.

Overall, the pipeline combines real-time data ingestion, ragged-edge handling, and recursive forecasting to produce timely and coherent GDP nowcasts. The final outputs include bridge model predictions across different flash stages, along with ADL and AR benchmark forecasts, all expressed as quarter-on-quarter GDP growth rates.

6.0 Challenges and Areas for Improvement

A challenge we encountered during development was the dependence on FRED API for live data retrieval. Internal server failures for certain series could interrupt preprocessing and delay the generation of live nowcast outputs, especially if the affected variables were required model inputs. To mitigate this, warning messages were added to flag missing series and prompt reruns. An area for future improvement would be to make the backend more robust to such disruptions through instant retries or cached fallback data.

A limitation of our bridge model is that it uses a fixed set of variables selected from a single LASSO run on data from 1960Q1 to 2020Q2, rather than being re-estimated as new observations are added to the sample for nowcasting. While this modeling choice improves pipeline stability, it assumes that the relevance of selected macroeconomic variables remain constant across time. However, in reality, the underlying relationship between predictors and GDP growth may change as additional data are released or as the economy moves across different regimes. Thus, there is a trade-off between stability and adaptability, where maintaining a fixed specification simplifies deployment but may reduce model performance if the most informative predictors shift over time.

A natural extension would be to re-run the LASSO feature selection procedure periodically as the dataset is updated, allowing the bridge model to refresh its specifications to cater to new information which can improve forecast accuracy and robustness.

Another limitation is that although the backend captures the progressive update of monthly information through three flashes nowcasts within each quarter, the bridge model converts the monthly indicators into quarterly aggregated predictions before estimating quarterly GDP growth. Hence, some intra-quarter information may be lost in final regression. In theory, this could affect forecast performance since the sequence and timing of monthly releases may contain information that is not fully preserved after aggregation.

To explore the extent to which the loss of intra-quarter information may affect forecast performances, we can benchmark our current bridge model against a Mixed Data Sampling (MIDAS) nowcasting model designed to incorporate monthly indicators directly when forecasting quarterly GDP growth. This allows us to test whether preserving more intra-quarter information can lead to improvements in flash nowcast performances.

Another possible application is the Diebold-Mariano test. Currently, we compare our models in terms of RMSE, MAE and Directional accuracy. The Diebold-Mariano test is a statistical test that goes a step further, allowing us to tell whether our bridge model's lower RMSE is statistically significant or not. This would provide stronger empirical evidence of the bridge model's superior nowcasting performance, and complement the descriptive RMSE, MAE and Directional accuracy metrics.

7.0 Responsibilities

Bryce:

- Data preprocessing (i.e. data cleaning pipeline, handling of missing values)
- LASSO implementation
- Rolling window tuning (i.e. conducted extensive hyperparameter tuning to determine optimal window_size, although later replaced by expanding window)
- Built Hybrid FRED API and CSV fallback pipeline into backtesting "execution.py" file
- Technical Documentation

- Presentation Slides and Project Presentation

Charlene:

- Set up and managed the Github repository and overall pipeline structure
- T-code transformations and monthly aggregation of FRED-MD indicators
- Implemented and evaluated a Random Forest benchmark model, including feature preparation and hyperparameter tuning
- Developed the flash nowcasting framework for the evaluation of bridge model
- Technical Documentation, Slides and Video

Sze Ping:

- Built and refined AR and ADL benchmark models
- Implemented ragged-edge handling using AR models
- Developed the live nowcasting pipeline for quarterly GDP prediction using latest data
- Set up the FRED API refresh pipeline to update monthly indicators and quarterly GDP data
- Structured model outputs into CSV files for frontend integration
- Technical Documentation and slides

Vanisha:

- Data preprocessing, Feature Selection, Technical Documentation, Video Script

8.0 Reflection

Although the project was originally scheduled to begin in Week 7, we started working on it towards the end of Week 8, which resulted in a more compressed development timeline than initially anticipated. An early milestone was the establishment of the AR and ADL benchmark models, which provided a baseline for evaluating the broader nowcasting framework.

As the project progressed, a significant milestone was the refinement of model logic following consultation in Week 10, where several weaknesses in the initial implementation were identified and corrected. In particular, this helped reveal specification issues in the backend pipeline, where lagged information needed to be treated carefully to prevent look-ahead bias, and to ensure that the historical and live nowcasts were generated consistently.

Finally, after stable and interpretable results have been obtained across benchmark and bridge models, the API component was completed in Week 11. This marked the transition from a historical modelling workflow to an actual live nowcasting pipeline.

Part III

9.0 General Layout and Theme

Our dashboard consists of three main components: live statistics, a monthly nowcast chart and a historical GDP chart. Each component is placed in a separate tab to clearly distinguish statistics and improve ease of navigation. The user can switch between tabs via the sidebar. Each component has a configuration panel, placed in the sidebar to allow the user to focus on the main display, allowing the user to configure the charts however they like.

By combining both current and historical statistics, the dashboard supports multiple analytical purposes. Live updates provide timely information for decision making, while historical charts help users identify business cycle patterns and assess broader economic trends.

10.0 Key UI Components and Functions

At the top right of our dashboard is a refresh button, which controls the interaction between the front-end and the back-end. The dashboard's data is stored in CSV files that are updated through a pipeline connected to the FRED database via the FRED API. When the user clicks the refresh button, this input is passed from the front-end to the back-end, which then retrieves the latest available predictor values, updates the relevant datasets, reruns the nowcasting models, and returns the updated outputs to the interface. The front-end then displays the refreshed statistics in the cards and charts. We chose to make the refreshing process more deliberate rather than automatic for both performance and practical reasons. Triggering an automatic refresh each time the user opens the dashboard will lead to longer load time, reduce responsiveness and risk unnecessary API usage. By allowing users to trigger the update only when required, the interface remains efficient while ensuring access to recent information.

10.1 Live Statistics

Refer to Figure 5 in the Appendix.

Our first tab, titled "Live Statistics", is designed to give users an immediate snapshot of the current state of the economy. As the default landing tab of the dashboard, it presents the most relevant real-time information upfront in a format that is quick and easy to interpret. This has two parts: statistical cards and a monthly nowcast chart beneath them. This layout is intentionally hierarchical, with the most important high-level information placed at the top and more detailed supporting information shown below.

At the top of the tab, users see 6 cards – a business cycle indicator card, the latest nowcast predictions from our AR, ADL and bridge models and cards containing the Atlanta Fed and St. Louis Fed nowcasts. Since cards are one of the most direct and readable UI elements, we used them to present a quick summary of current-quarter economic conditions and statistics that users are mostly likely to look for first, making the interface more intuitive.

The business cycle indicator provides an additional layer of depth beyond numerical nowcasts. It translates the GDP estimates into a more informative signal of the broader state of the economy: expansion, decelerating growth, recession or contraction. To make this easier to interpret, the indicator uses colour coding as a visual cue. Green represents an expansion and signals a positive business cycle outlook, yellow represents decelerating growth and signals moderation, while red indicates recession or contraction and highlights a weaker economic state. This design choice improves usability by allowing users to understand the economic context immediately, even before reading exact figures.

The rationale behind placing cards for 3 of our models, together with that of external nowcasts is so that the main bridge prediction can be compared across the overall values. If there is an overall consensus of the nowcast direction, then the user will know that the bridge model prediction is mostly reliable and does not deviate too much from these benchmark models.

Among these cards, the bridge model nowcast is given the greatest visual prominence, through pulsating effects. This design decision reflects how the bridge model serves as our main nowcasting model. This is because it is best suited to the mixed-frequency data, where quarterly GDP is predicted using monthly indicators. It is also more robust in providing a nowcast because it can still produce nowcasts even when some monthly data for the current quarter is not available yet. As new monthly indicators are released, the bridge model incorporates this information quickly, allowing it to respond more effectively to current economic developments. For this reason, we placed the live bridge nowcast first so that the user's attention is drawn to it first.

Below the cards, there is a monthly chart showing the evolution of the bridge model nowcast for the current quarter. This chart adds a dynamic element to the interface by showing how the nowcast changes as more monthly predictor data becomes available over time. This is useful in our dashboard,

since the estimate is not fixed but evolves as the data improves throughout the quarter. Presenting this progression visually helps users understand the responsiveness of the mode and gives them a record of how the estimate has changed in response to newly released indicators. The monthly chart also contains 50% and 80% prediction intervals which can be toggled on or off, to show the user the positive and negative outlooks of each monthly nowcast.

10.2 Monthly Nowcast

Refer to Figure 6 in the Appendix.

The second tab of our dashboard, titled "Monthly Nowcast", is designed to give users a more flexible and detailed view of how the nowcast evolves within a quarter. While the monthly chart in the "Live Statistics" tab only focuses on the current quarter, this tab allows users to select historical quarters and examine the intra-quarter evolution of the bridge model for those periods. Therefore, the "Monthly Nowcast" tab goes beyond a live view and provides an interactive tool for retrospective analysis.

Once a quarter is selected, this input is passed to the back-end, which retrieves the corresponding monthly nowcast values from our CSV file which stores historical monthly evolution data. This data is then returned to the front-end for the visualisation. The chart displays the nowcast for each of the three months within the selected quarter, capturing the complete progression of the estimate, including the stage at which the nowcast is based on the most complete set of information. This intra-quarter bridge model evolution chart gives the user control over the time period being shown, making the chart more useful for users to understand how the model behaved across different economic conditions.

To strengthen interpretation, the chart also has an actual GDP dotted line layered over it. This enables users to compare the sequence of monthly nowcasts against the actual GDP outcome for that quarter. This shows how the accuracy of the prediction changed as more monthly data became available, as the user can assess how close the nowcast was to the actual observed value. Thus, the path of the model's predictions can be contextualised with the true economic outcome.

10.3 History Chart

Refer to Figure 7 in the Appendix.

The third tab in our dashboard, called "History Chart", is designed to provide users with a broader historical view of GDP and nowcast performance over time. Unlike the first two tabs, which focus on current conditions and intra-quarter changes, this tab allows for more comprehensive analysis by plotting actual GDP together with nowcasts from our AR, ADL and bridge models, as well as external benchmark models from the Atlanta Fed and St. Louis Fed, across quarters. Its main purpose is to help users compare model predictions across time and identify broader economic trends and business cycle patterns.

The historical comparison chart has a model configuration panel, found in the sidebar. The configuration panel allows users to customise what is shown on the chart. The user can select which nowcast series they want to include in the graph. They can also select the year, quarter and number of quarters to be displayed. These user selections are captured by the front-end and passed to the back-end, where there are multiple CSV files storing the historical data for each model. As for the Atlanta Fed and St. Louis Fed data, it is retrieved via API. The requested lines are then returned to the front-end for display. This interaction makes the chart more flexible, since different users may be interested in different combinations of models depending on whether they want to compare internal models, evaluate performance against external benchmarks, or focus on one model's trend over time. Furthermore, when the user selects a specific quarter, that quarter is centred in the chart, making the surrounding quarters visible too. This design choice was made to provide context rather than isolating a single quarter, which is important for identifying trends and business cycles. The user can also increase or reduce the number of quarters displayed to obtain a wider or narrower historical view, depending if they want to focus on short-run or long-run trends.

The actual historical GDP line is fixed and cannot be removed, as it is a baseline reference for comparison. It is also the only line displayed as a solid line, while the others are displayed as dotted lines. This visually reinforces that actual GDP is a baseline for comparisons. We also included the Atlanta Fed and St. Louis Fed nowcasts to serve as an external benchmark for comparison, allowing users to evaluate the performance of our models against other well-established models. Moreover, when the user opens the graph, the default lines that are displayed are the AR and bridge graphs. This is because the bridge model is our main model and the AR model is our benchmark model, serving as a good comparison.

Since the chart contains multiple lines at once, readability was a crucial consideration. To address the issue of cluttering, we incorporated hover functionalities, allowing users to move their cursor over each point to view the exact value of each model at any quarter. This improves readability instead of overcrowding the chart with labels. Furthermore, the model configuration panel was included to reduce clutter by letting users display the lines that are most relevant to them.

11.0 Back-End to Front-End Pipeline

11.1 Back-End Databases

11.1.1 FRED Industry Models

In order to amass the historical nowcast from Atlanta Fed's GDPNOW and St Louis Fed models, we made use of the API key from the FRED to pull the nowcasts by year and quarter. Using a Streamlit-cached function to minimize redundant network calls, it retrieves three key series: actual Real GDP (GDPC1), the Atlanta Fed's GDPNow forecast, and the St. Louis Fed's STLENI forecast. As raw Real GDP is reported in absolute levels, the script applies a transformation to convert it into a Quarter-over-Quarter Annualized Rate, aligning it perfectly with the percentage-based Fed nowcasts. Finally, it aligns all series to a standardized quarterly frequency, specifically forcing the current quarter's index to reflect the absolute latest live value. With that, we used *fred_industry_models.py* to build a dataframe of the historical nowcasts from these industry nowcast models to be visualized in our historical chart.

11.1.2 AR, ADL and Bridge Models

Within our historical chart, we aim to present the historical nowcasts of all our models given the historical data and indicators available at that time. To achieve that, we ran *export_adl_history.py*, *export_ar_history.py* and *export_bridge_history.py* to conduct an expanding-window simulation across all historical data, providing a robust database for evaluation. Across the Autoregressive (AR), Autoregressive Distributed Lag (ADL), and Bridge models, the core pipeline mechanism relies on stepping chronologically through the dataset and dynamically re-training the equations using only information available prior to the target quarter, thereby strictly preventing look-ahead bias. While the AR and ADL benchmark pipelines utilize clean, lagged quarterly variables to generate 1-step ahead retrospective baselines, the Bridge pipeline handles the complex "ragged edge" of real-time macroeconomic reporting by simulating sequential monthly data releases and imputing missing intra-quarter observations.

11.1.3 Bridge Intra-Quarter

In order to produce the intra-quarter database for our Bridge model, we simulate the historical flow of macroeconomic data releases by dividing each target quarter into sequential "Flashes" to account for publication lags. As these varying release schedules create a "ragged edge" of missing observations at the end of the sample, we dynamically train Autoregressive (AR) models to forecast and impute the missing monthly data. Once the dataset is fully populated, we aggregate these high-frequency indicators to a quarterly level, append necessary structural variables (like a COVID-19 dummy) and historical GDP lags, and feed the complete matrix into our primary OLS Bridge equation. By iterating this process chronologically and strictly isolating training data to the previous quarter to prevent

look-ahead bias, the pipeline generates an evolutionary timeline of predictions that accurately reflects real-time model performance as new information becomes available.

11.1.4 Live-API Monthly and Quarterly GDP

To ensure our nowcasts remain continuously up-to-date, we developed api_preprocessing.py to automate the ingestion of live economic data. This script triggers a bulk data pull from 1960 to the present day, retrieving all viable monthly indicators and automatically renaming the columns to match our project's standardized nomenclature for seamless downstream compatibility. Additionally, the script fetches the quarterly Real GDP target variable (GDPC1) and exports the datasets into live_api_monthly.csv and live_api_quarterly_gdp.csv. This automated pipeline guarantees that our models consistently train and evaluate on the most current macroeconomic reality.

11.2 Components and Data integration

In the table below, we provide a high level mapping of where each component draws its data from.

Component	Database
Business Cycle	live_nowcast_results.csv
Nowcast Metric Cards a. Bridge Model b. AR Model c. ADL Model	live_nowcast_results.csv
Models Metric Cards a. Atlanta GDPNOW b. St Louis Fed	API call to FRED
Intra-Quarter Chart	bridge_evolution.csv
Live Graph	live_nowcast_results.csv
History Chart	historical_gdp_ar_predictions.csv historical_gdp_adl_predictions.csv historical_gdp_bridge_predictions.csv

11.3 End-to-End Data Integration Pipeline

Refer to Figure 8 in the Appendix.

The application utilizes a decoupled architecture, separating heavy backend data processing from the frontend Streamlit presentation layer to ensure a responsive user experience. Rather than holding massive datasets in Streamlit's session state, the architecture relies on a series of intermediate CSV files (e.g. live_nowcast_results.csv, bridge_evolution.csv) acting as the "database." At the presentation layer, the frontend integrates the static predictions and live API calls from the FRED API to render the final interactive dashboard.

12.0 Limitations and Possible Improvements

While Streamlit enables rapid prototyping and straightforward cloud deployment, its top-down execution model introduces constraints for building complex, stateful applications. In particular, managing asynchronous workflows, such as triggering and persisting live FRED API refreshes across multiple navigation tabs, requires non-trivial workarounds (e.g., session state management), and can lead to unintended UI re-renders or silent state resets.

In addition, Streamlit offers limited flexibility in frontend customization. Although basic styling can be achieved through custom HTML/CSS injection, it remains difficult to implement fully customized, production-grade user interfaces with consistent branding and fine-grained control over interactivity.

A natural next step would be to decouple the frontend and backend architecture. Migrating the frontend to a dedicated framework such as React (e.g., using Next.js) would allow for more robust state management, asynchronous data handling, and richer UI/UX customization. Streamlit could then be retained as a lightweight internal prototyping tool, while the production system adopts a more scalable, modular architecture.

Beyond the frontend architecture, the terminal's real-time functionality is inherently dependent on the reliability of the Federal Reserve Economic Data (FRED) API. As a result, upstream issues—such as latency, rate limiting, or unexpected data revisions—can directly impact the responsiveness and accuracy of the nowcast. In such cases, the backend preprocessing pipeline may be delayed while resolving these external constraints, which can affect the timeliness of updates presented in the UI.

To mitigate this, we implemented caching and controlled data refresh logic to reduce redundant API calls and handle rate limits more gracefully. However, the system remains ultimately constrained by the availability and consistency of upstream data. Future improvements could explore integrating alternative data sources or maintaining a local mirrored database to further reduce reliance on a single external provider.

13.0 Potential Extensions

13.1 Economic Shock Simulator

This feature would allow users to simulate macroeconomic shocks (e.g. a spike in unemployment) and observe their impact on the nowcast in real time. Interactive sliders could be integrated into the live statistics page, enabling users to adjust the values of individual indicators used in the Bridge model. As these inputs are modified, the system would dynamically update the GDP nowcast, providing immediate feedback on how changes in key economic variables influence overall economic activity.

13.2 Monthly Newsletter

Users will be able to register their email to receive an automated monthly newsletter summarizing changes in the GDP nowcast. At the end of each month, the system would generate and send a concise update highlighting key movements in the nowcast, along with brief insights into the underlying drivers.
This feature allows users to stay informed without needing to actively monitor the dashboard, providing a convenient and timely overview of evolving economic conditions.

13.3 Results Synthesis

Additional cards or dropdowns could present some evaluations to make meaning of the statistics. For example, the accuracy of nowcast compared to actual GDP. Users could be allowed to add annotations to the graphs such as the economic events that affected GDP, to help them more clearly understand the context of the data. This helps the user to synthesize the results for use.

14.0 Team Contributions

Javier:

- Overall user interface aesthetics and design in terms of colour scheme, theme and features layout, including but not limited to sidebar, configurations, and graphs dimensions
- Developed the FRED models pipeline for live metrics and history chart
- Implementation of tooltips for metric cards to provide additional information without cluttering the dashboard
- Implementation of refresh button to rerun the entire pipeline in real-time
- Added visual effects on 'Business Cycle' and 'Bridge Nowcast' metrics card
- Designed the team logo
- Technical documentation

Melanie:

- Created the live metric and business cycle cards

- Overall user interface design

Owen:

- Designed the interactive configuration panel to manage user inputs and model parameters.
- Engineered the real-time monthly nowcast dashboard and a live evolution graph.
- Extraction of Standard Errors (SE) from the backend to accurately represent statistical uncertainty in the UI through the prediction intervals.
- Developed dynamic Fan Charts and custom candlestick error bars in Plotly to clearly visualize prediction intervals.
- Constructed the robust data pipeline powering the bridge model's current evolution and monthly nowcast visualizations.
- Technical documentation

Shannon:

- Implemented history chart (AR, ADL, Bridge, Actual GDP) and year, quarter and number of quarter configurations
- Developed historical data pipeline for bridge, AR and ADL models to be integrated into history chart
- Added the recent 2026 Q1 nowcasts into monthly and historical charts
- Debugged data accuracy errors in Atlanta Fed live metric
- Refined user interface design of cards and history chart
- Technical documentation

15.0 Project Journey (Timeline, Milestones, Deviations from Proposal)

Our project timeline deviated significantly from our original proposal. We initially planned to start the dashboard development in week 10, after the models were completed. However, as we needed to work closely with the back-end team, support one other across tasks, and re-evaluate the features to include, we began dashboard work earlier in week 9.

This meant our workflow became less sequential than planned. We originally expected to complete the data pipelines first and then enhance the user interface. However, in reality, we designed the first version of the user interface before the data pipelines were ready, since we were waiting for the back-end models to be completed. Starting with the interface gave us a strong foundation to build on once the models were available.

A key milestone was receiving the AR benchmark model from the back-end team. This allowed us to begin integrating real model outputs into the dashboard and test whether the interface worked with actual data. From there, we exported the relevant CSV files and integrated the remaining models. This stage took longer than expected because connecting the back end to the front end involved multiple Python scripts and data files, which made the process more complex.

The final stage focused on refinement. Once the main features were in place, we improved the dashboard’s aesthetics, usability, and overall user experience, while also addressing bugs and incorporating feedback from the back-end team.

Overall, although our process differed from the original plan, we adapted our timeline and workflow to the project’s needs and were still able to deliver a viable final product on time.

Appendix

Figure 2:

Appendix

The column TCODE denotes the following data transformation for a series x: (1) no transformation; (2) Δx_t; (3) $\Delta^2 x_t$; (4) $log(x_t)$; (5) $\Delta \log(x_t)$; (6) $\Delta^2 \log(x_t)$; (7) $\Delta(x_t/x_{t-1} - 1.0)$. The FRED column gives mnemonics in FRED followed by a short description. The comparable series in Global Insight is given in the column GSI.

Some series require adjustments to the raw data available in FRED. We tag these variables with an asterisk to indicate that they been adjusted and thus differ from the series from the source. A summary of the adjustments is detailed in the paper https://research.stlouisfed.org/wp/2015/2015- 012.pdf

Figure 3:

Figure 4:

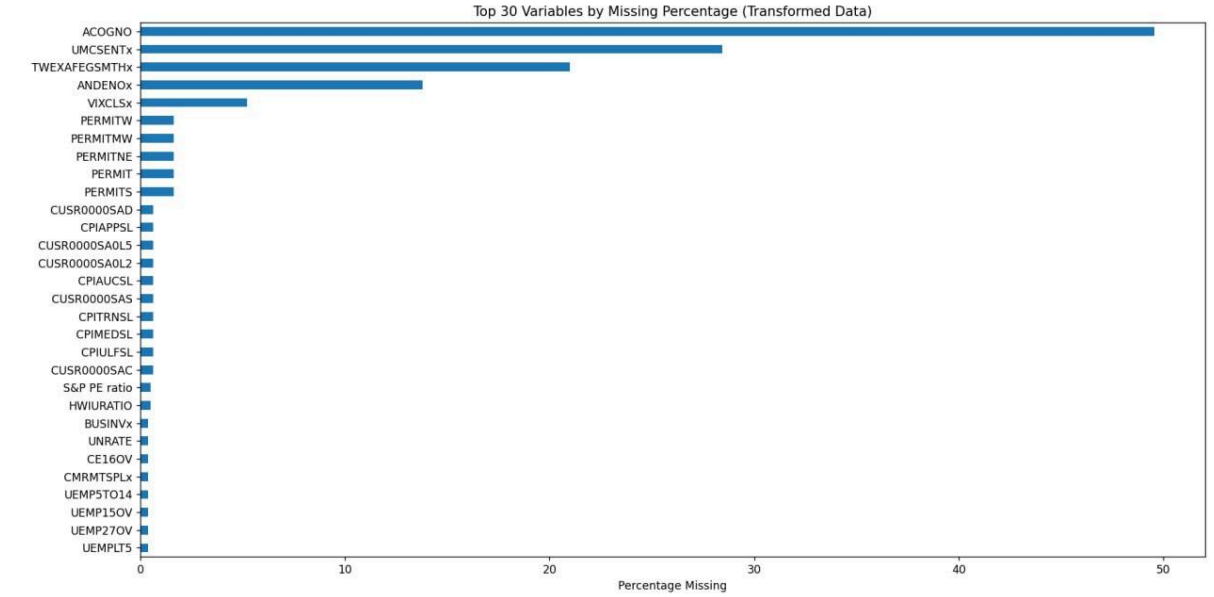


Figure 5:

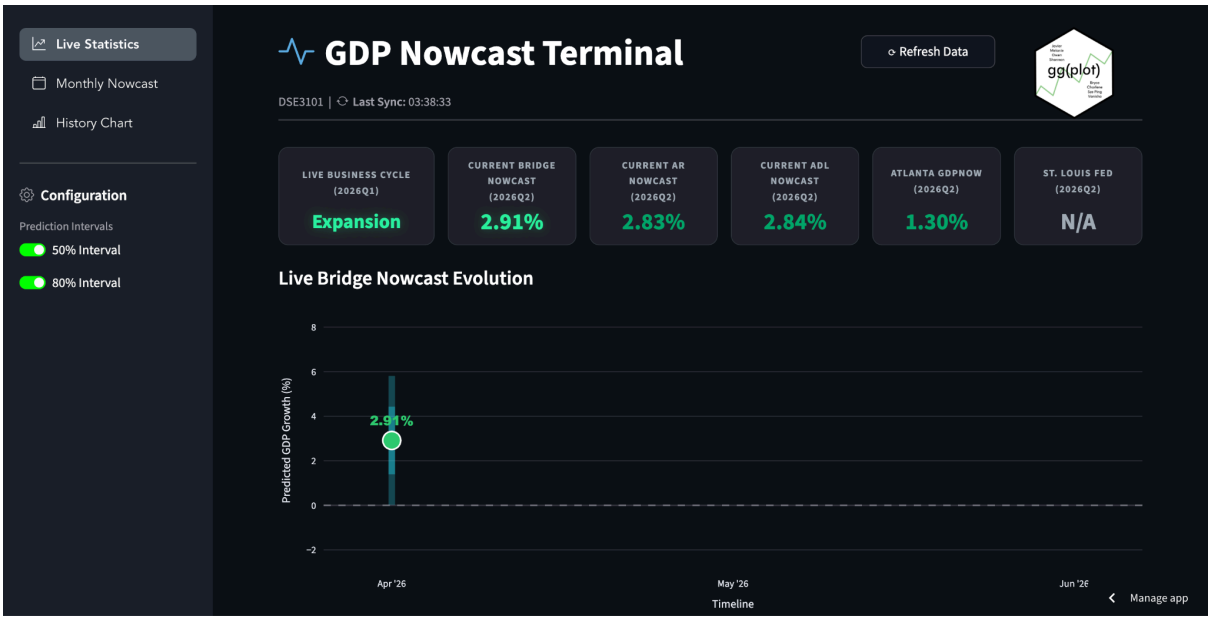


Figure 6:

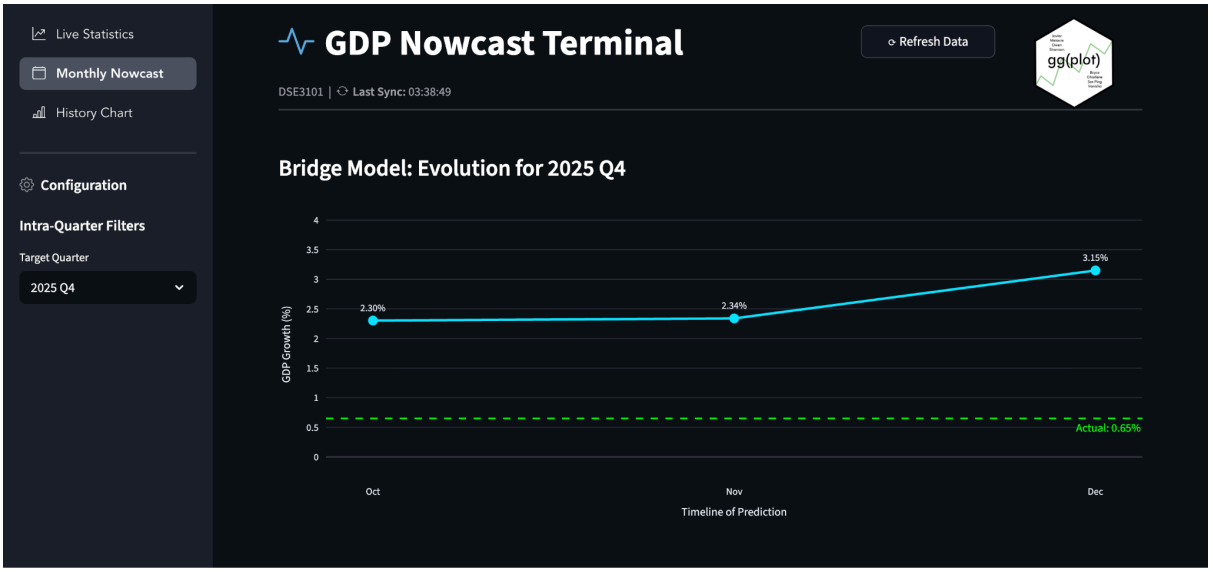


Figure 7:

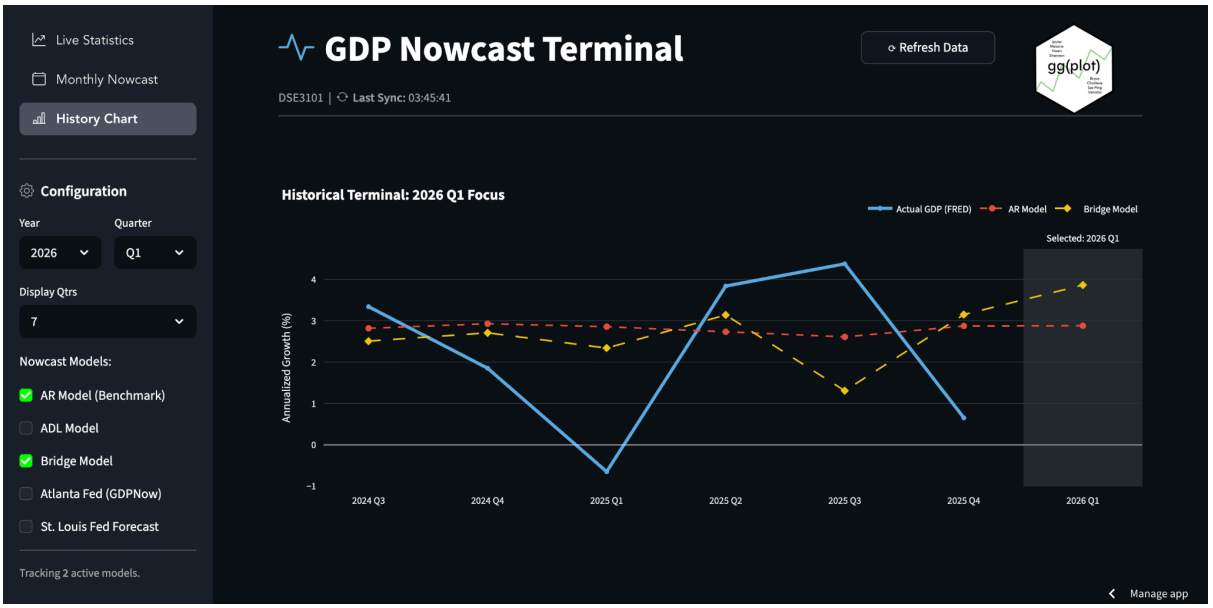
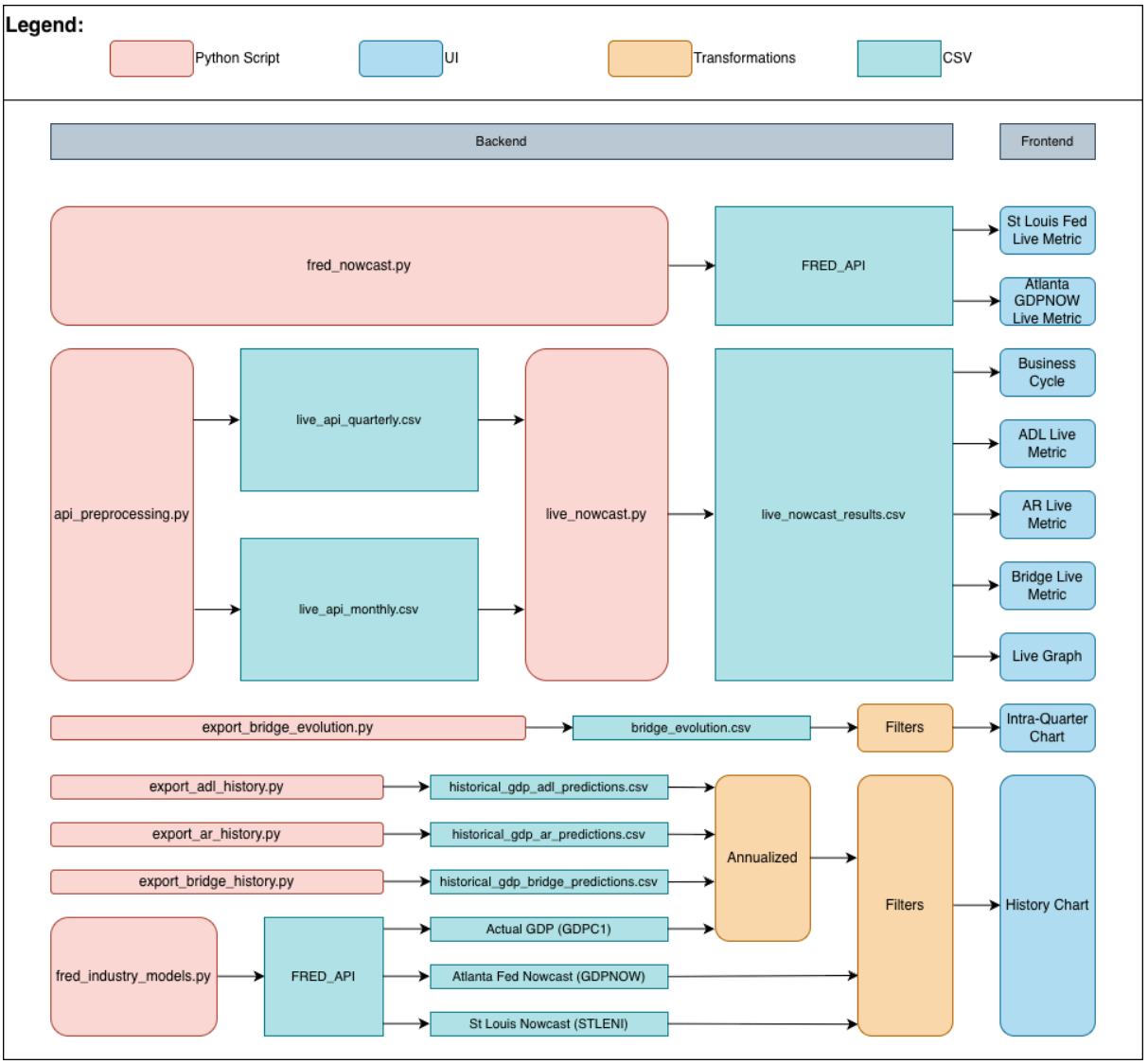

Live Statistics
Monthly Nowcast
History Chart
Configuration
Year
Quarter
2026
Q1
Display Qtrs
7
Nowcast Models:
AR Model (Benchmark)
ADL Model
Bridge Model
Atlanta Fed (GDPNow)
St. Louis Fed Forecast
Tracking 2 active models.
GDP Nowcast Terminal
Refresh Data
DSE3101 | Last Sync: 03:45:41
Historical Terminal: 2026 Q1 Focus
Actual GDP (FRED)
AR Model
Bridge Model
Selected: 2026 Q1
Annualized Growth (%)
4
3
2
1
0
−1
2024 Q3
2024 Q4
2025 Q1
2025 Q2
2025 Q3
2025 Q4
2026 Q1
Manage app

Figure 8:


Legend:
Python Script
UI
Transformations
CSV
Backend
Frontend
fred_nowcast.py
FRED_API
St Louis Fed Live Metric
Atlanta GDPNOW Live Metric
api_preprocessing.py
live_api_quarterly.csv
live_api_monthly.csv
live_nowcast.py
live_nowcast_results.csv
Business Cycle
ADL Live Metric
AR Live Metric
Bridge Live Metric
Live Graph
export_bridge_evolution.py
bridge_evolution.csv
Filters
Intra-Quarter Chart
export_adl_history.py
export_ar_history.py
export_bridge_history.py
historical_gdp_adl_predictions.csv
historical_gdp_ar_predictions.csv
historical_gdp_bridge_predictions.csv
Annualized
Filters
History Chart
fred_industry_models.py
FRED_API
Actual GDP (GDPC1)
Atlanta Fed Nowcast (GDPNOW)
St Louis Nowcast (STLENI)